Дисциплина: Фреймворки
Практическая работа № 7. Базы данных: язык SQL и работа с SQLite из Python.
Группа: ЭФБО-10-24
ФИО: Сизова Софья Ивановна
Год выполнения: 2026

1. Общие сведения

- Название практической работы: ПР7. SQL и SQLite.
- Дисциплина: Фреймворки
- Группа: ЭФБО-10-24
- ФИО: Сизова Софья Ивановна
- Индивидуальный проект: «Сервис управления подписками на уведомления» (NotifyHub)
- Репозиторий: https://github.com/sonic0666/fr1

2. Выполненные материалы Яндекс Практикума

Раздел	Выполненные темы	Выполнено
Базы данных и SQL	Реляционные БД; таблицы, ключи, связи; создание таблиц; `INSERT`, `SELECT`, `WHERE`; сортировка и агрегаты; `GROUP BY`, `HAVING`; `JOIN`; подзапросы; модуль `sqlite3`	□

Отметь пункты самостоятельно после прохождения материалов на платформе.

3. Доработка индивидуального проекта

Данные веб-интерфейса перенесены из JSON-файлов в базу данных SQLite `data/notifyhub.sqlite`.

Структура базы (`sql/schema.sql`):

Таблица	Поля и ограничения
`notification_types`	`id` — PRIMARY KEY; `name` — NOT NULL
`subscribers`	`id` — PRIMARY KEY; `name`, `email` — NOT NULL
`subscriptions`	`id` — PRIMARY KEY; `subscriber_id`, `type_id` — FOREIGN KEY на `subscribers` и `notification_types`; `channel` — CHECK (email / push / sms); `end_date`; `is_active` — DEFAULT 1, CHECK (0 или 1)

Связи: один подписчик — много подписок, один тип — много подписок. Внешние ключи включаются командой `PRAGMA foreign_keys = ON` при каждом подключении.

Выполненная работа:

- создан пакет `db/`: `database.py` (подключение, `row_factory = sqlite3.Row`), `import_json.py` (перенос данных из JSON в таблицы через `executemany`, в одной транзакции с `commit`/`rollback`, родительские таблицы первыми), `queries.py` (SQL-запросы веб-интерфейса);
- написан файл `sql/queries.sql` с 15 запросами: выборка, `WHERE`, `LIKE`, `BETWEEN`, `ORDER BY`, `JOIN` трёх таблиц, подзапрос `NOT IN`, `COUNT`, `MIN`/`MAX`, `GROUP BY`, `LEFT JOIN`, `HAVING`, `LIMIT`/`OFFSET`;
- все значения передаются в запросы параметрами `?`, а не подстановкой в строку, что исключает SQL-инъекции;
- view-функции главной страницы, типов и подписок переведены на SQLite;

- добавлен фильтр списка подписок по каналу;
- добавлена страница «Доступные типы» (`/types/available/`): выбранный подписчик и дата, запрос с подзапросом `NOT IN`, проверка входных данных (код 400 при ошибке);
- добавлена страница «Статистика» (`/subscriptions/statistics/`): подписки по типам (`LEFT JOIN` + `GROUP BY`), по каналам, популярные типы (`HAVING`);
- главная страница показывает количество записей в таблицах;
- тесты используют временную БД (`tmp_path`) с тестовыми данными: проверяются схема, ограничения, внешние ключи, импорт и откат, выполнение всех запросов из `queries.sql`, страницы и защита от SQL-инъекции; всего 86 тестов;
- обновлён `README.md` (таблицы, ключи, запуск, запросы).

4. Результат работы

- база создаётся из `sql/schema.sql` и заполняется командой `python -m db.import_json`, количество записей в БД совпадает с JSON (3 типа, 2 подписчика, 3 подписки);
- нарушение внешнего ключа или ограничения `CHECK` приводит к ошибке `IntegrityError`;
- веб-интерфейс работает с данными из SQLite;
- консольная версия (`main.py`) продолжает использовать JSON;
- результат зафиксирован в Git коммитами с меткой `PR7:`; `pytest`, `flake8` и `manage.py check` проходят без замечаний.

5. Вывод

В ходе ПР7 спроектирована реляционная база данных проекта с первичными и внешними ключами и ограничениями целостности, освоены основные конструкции SQL (выборка, фильтрация, соединения, группировка, подзапросы) и работа с SQLite из Python через модуль `sqlite3`. Параметризованные запросы обеспечивают безопасность, а транзакции — целостность данных при импорте. Проект подготовлен к переходу на Django ORM.